**Name : T Bhanu Teja**
**Reg no: 192472259**

# UNIT I

# PRINCIPLES OF OBJECT-ORIENTED PROGRAMMING & CONTROL STRUCTURES

*C++ Programming – Questions with Answers*

Syllabus scope: OOP concepts and paradigms; tokens, keywords, identifiers and constants; data types and variables; operators; type conversion and casting; precedence and associativity; selection, iteration and jump statements.

## Assessment Answer Format

ssSection A – Debugging: Each answer identifies the defective line and error category, explains the root cause, provides complete corrected C++ code, and includes a dry run with the predicted output.

Section B – Pseudocode: Each answer includes a brief design overview, structured pseudocode using the prescribed uppercase keywords and explicit data types, and a trace table using the sample input.

# SECTION A: DEBUGGING QUESTIONS

Each debugging question is presented with its topic, problem statement, original buggy code, a brief answer introduction, corrected C++ code, dry run, and exact output.

## Question 1: Operator Precedence Trap with Bitwise Operations & Implicit Promotion

**Topic:** Operators in C++, operator precedence, implicit conversion.

**Problem Statement:** A student intends to check whether an unsigned 8-bit integer is odd and greater than 10. The buggy condition is `if (num & 1 == 1 && num > 10)`. The program also counts down from 3 to 0 using an unsigned loop counter. The supplied code uses `for (unsigned char i = 3; i >= 0; --i)` and a temporary `if (i == 0) break;` patch.

**Buggy Code:**

```
#include <iostream>

int main() {
    unsigned char num = 15;

    // Line 7: Intended to check if num is odd AND greater than 10
    if (num & 1 == 1 && num > 10) {
        std::cout << "Valid Odd Number: " << (int)num << std::endl;
    } else {
        std::cout << "Condition Failed for: " << (int)num << std::endl;
    }

    // Line 14: Intended to decrement from 3 down to 0 inclusive
    for (unsigned char i = 3; i >= 0; --i) {
        std::cout << (int)i << " ";
        if (i == 0) break; // Added as a temporary patch
    }
    std::cout << std::endl;

    return 0;
}
```

**ANSWER**

The errors are identified below, followed by the corrected C++ program, a step-by-step dry run, and the exact output.

**1. Line number and error classification**

- Condition line (`if (num & 1 == 1 && num > 10)`): Logic defect caused by operator precedence. Equality (`==`) has higher precedence than bitwise AND (`&`), so the expression is grouped as `num & (1 == 1)` rather than `(num & 1) == 1`.
- Countdown loop (`unsigned char i = 3; i >= 0; --i`): Logic defect / non-terminating loop risk. An unsigned value cannot become negative; decrementing zero wraps to its maximum representable value. The temporary break avoids the wrap only while it remains present.

**2. Root cause**

The intended odd-number test must first isolate the least significant bit using `num & 1`, then compare that result with 1. The loop condition `i >= 0` is always true for an unsigned type. Use a signed integer counter for a countdown that includes zero.

**3. Corrected C++ code**

```
#include <iostream>

int main() {
    unsigned char num = 15;

    if ((num & 1) == 1 && num > 10) {
        std::cout << "Valid Odd Number: "
                  << static_cast<int>(num) << std::endl;
    } else {
        std::cout << "Condition Failed for: "
                  << static_cast<int>(num) << std::endl;
    }
```

```
    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

**4. Dry run and output**

| Step | Value / evaluation | Result |
|---|---|---|
| Odd test | 15 & 1 = 1; 1 == 1; 15 > 10 | Both conditions true |
| Loop 1 | i = 3 | Print 3 |
| Loop 2 | i = 2 | Print 2 |
| Loop 3 | i = 1 | Print 1 |
| Loop 4 | i = 0 | Print 0; next i becomes -1 and loop stops |

Exact output:

```
Valid Odd Number: 15
3 2 1 0
```

## Question 2: Scope Resolution & Variable Shadowing Conflicts

**Topic:** Variables, scope resolution operator (`::`), constants.

**Problem Statement:** An e-commerce invoicing system has a global `double taxRate = 0.18`. Inside `main()`, a local `taxRate = 0.05` shadows it. In an inner block, an integer `taxRate = 12` shadows the local variable. The buggy code uses `:taxRate` in the global calculation and `::taxRate` where it intends to access the outer local rate.

**Buggy Code:**

```
#include <iostream>

double taxRate = 0.18; // Global Standard Tax Rate (18%)

int main() {
    double itemPrice = 1000.0;
    double taxRate = 0.05; // Local Subsidized Rate (5%)

    std::cout << "Local Rate: " << taxRate << std::endl;

    // Line 11: Developer attempts to use the global tax rate
    double totalWithGlobalTax = itemPrice + (itemPrice * :taxRate);

    {
        int taxRate = 12; // Nested local variable shadowing

        // Line 16: Developer attempts to access the outer local rate (0.05)
        double totalWithLocalTax = itemPrice + (itemPrice * ::taxRate);
        std::cout << "Inner total: " << totalWithLocalTax << std::endl;
    }

    std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
    return 0;
}
```

**ANSWER**

The errors are identified below, followed by the corrected C++ program, a step-by-step dry run, and the exact output.

**1. Line number and error classification**

- `itemPrice * :taxRate`: Compilation error. The global scope operator must be written as `::`, not `:`.

- `::taxRate` inside the nested block refers to the global variable, not the outer local variable. The scope resolution operator accesses a name in global/namespace scope; it cannot select an enclosing local variable.

**2. Root cause**

The inner `int taxRate` hides the outer local `double taxRate` within its block. C++ has no syntax equivalent to `outer::taxRate` for an enclosing function-local variable. The clean solution is to give the local rate a distinct name.

**3. Corrected C++ code**

```
#include <iostream>

double taxRate = 0.18; // Global standard rate

int main() {
   double itemPrice = 1000.0;
   double localTaxRate = 0.05;

   std::cout << "Local Rate: " << localTaxRate << std::endl;

   double totalWithGlobalTax =
      itemPrice + (itemPrice * ::taxRate);

   {
      int corporateTaxRate = 12;
      double totalWithLocalTax =
         itemPrice + (itemPrice * localTaxRate);

      std::cout << "Inner total: "
            << totalWithLocalTax << std::endl;
      std::cout << "Corporate rate: "
            << corporateTaxRate << std::endl;
   }

   std::cout << "Final Global Calculation: "
         << totalWithGlobalTax << std::endl;

   return 0;
}
```

**4. Expected output**

```
Local Rate: 0.05
Inner total: 1050
Corporate rate: 12
Final Global Calculation: 1180
```

## Question 3: Switch-Case Variable Initialization & Illegal Case Types

**Topic:** Control structures (`switch-case`), tokens, data types.

**Problem Statement:** A lexical scanner uses `switch (tokenType)`. The buggy code declares `int offset` in case 1 and `int multiplier` in case 2 without separate case scopes, omits a `break` after case 2, and contains the floating-point label `case 4.5:`.

**Buggy Code:**

```
#include <iostream>

int main() {
   int tokenType = 2;
   float tokenValue = 0.0f;

   switch (tokenType) {
      case 1:
         int offset = 10;
```

```
            tokenValue = 50.0f + offset;
            break;

        case 2:
            int multiplier = 5; // Error occurs here!
            tokenValue = 20.0f * multiplier;
            // Developer forgot break statement here

        case 3:
            tokenValue += 100.0f;
            break;

        case 4.5: // Error occurs here!
            std::cout << "Float token detected" << std::endl;
            break;

        default:
            std::cout << "Unknown token" << std::endl;
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

**ANSWER**

The errors are identified below, followed by the corrected C++ program, a step-by-step dry run, and the exact output.

**1. Error classification and root cause**

- Cross-initialization: A switch can jump directly to a case label. Without braces, control may jump past the initialization of a local variable whose scope extends across later labels. Enclose declarations in their own `{ }` case blocks.
- Illegal case label: A `switch` condition uses an integral or enumeration type (after integral promotions); case labels must be converted constant expressions of a compatible integral/enum type. A floating-point value such as `4.5` is not permitted.
- Missing `break`: Case 2 falls through into case 3, adding 100 unexpectedly.

**2. Corrected C++ code**

```
#include <iostream>

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }

        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }

        case 3: {
            tokenValue += 100.0f;
            break;
        }

        case 4: {
            std::cout << "Integer token 4 detected" << std::endl;
            break;
        }
```

```
        default:
            std::cout << "Unknown token" << std::endl;
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

**3. Dry run and output**

For `tokenType = 2`, case 2 executes. `multiplier = 5`, so `tokenValue = 20.0 × 5 = 100.0`. The `break` prevents execution of case 3.

```
Calculated Value: 100
```

## Question 4: Reference Variable Initialization & Constant Binding Violations

**Topic:** Reference variables, constants, implicit conversions.

**Problem Statement:** A program attempts four reference operations: declares `int &refA;` without initialization, binds `int &refLiteral = 100;`, binds `int &refInt = pi;` where `pi` is a `double`, and assigns to `const int &constRef = 50;`.

**Buggy Code:**

```
#include <iostream>

int main() {
    int a = 25;

    // Line 7: Reference declaration without immediate binding
    int &refA;
    refA = a;

    // Line 11: Non-const lvalue reference bound to an rvalue literal
    int &refLiteral = 100;

    // Line 14: Non-const reference bound to an expression requiring conversion
    double pi = 3.14159;
    int &refInt = pi;

    // Line 17: Modification through a const reference
    const int &constRef = 50;
    constRef = constRef + 10;

    std::cout << a << " " << refA << std::endl;
    return 0;
}
```

**ANSWER**

The errors are identified below, followed by the corrected C++ program, a step-by-step dry run, and the exact output.

**1. Error classification and root cause**

- `int &refA;`: Compilation error. A reference must be initialized when it is declared.
- `int &refLiteral = 100;`: Compilation error. A non-const lvalue reference cannot bind to an rvalue literal.
- `int &refInt = pi;`: Compilation error. A non-const `int` reference cannot bind to a `double` object; it also cannot bind to a converted temporary.
- `constRef = constRef + 10;`: Compilation error. The object is accessed through a const reference and cannot be modified through that reference.

**2. Const-reference lifetime extension**

A const lvalue reference can bind to a temporary value. When a local const reference is directly bound to a temporary, the temporary's lifetime is extended to the lifetime of that reference. The reference still cannot be used to modify the temporary.

**3. Corrected C++ code**

```
#include <iostream>

int main() {
    int a = 25;
    int &refA = a;                // Valid alias

    const int &refLiteral = 100;   // Const reference to temporary

    double pi = 3.14159;
    double &refPi = pi;           // Same type, valid reference

    const int &constRef = 50;      // Temporary lifetime is extended

    std::cout << a << " " << refA << std::endl;
    std::cout << refLiteral << " " << refPi << std::endl;
    std::cout << constRef << std::endl;

    return 0;
}
```

**4. Expected output**

```
25                                                                                                  25
100                                                                                               3.14159
50
```

## Question 5: Dynamic Array Allocation & Stray Loop Semicolon

**Topic:** Memory operators (`new`, `delete`), loops, off-by-one boundaries.

**Problem Statement:** A telemetry processor allocates an array of five integers using `new int[size]`. The initialization loop has a stray semicolon after the `for` statement, and the array is released using `delete readings`.

**Buggy Code:**

```
#include <iostream>

int main() {
    int size = 5;

    // Line 7: Allocating dynamic array
    int *readings = new int[size];

    // Line 10: Loop initialization with a punctuation defect
    for (int i = 0; i < size; ++i);
    {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }
    std::cout << "Total: " << total << std::endl;

    // Line 21: Heap memory deallocation
    delete readings;

    return 0;
}
```

**ANSWER**

The errors are identified below, followed by the corrected C++ program, a step-by-step dry run, and the exact output.

**1. Error classification and root cause**

- Stray semicolon: `for (int i = 0; i < size; ++i);` makes the loop body an empty statement. The following block is separate from the loop.
- Scope issue: `i` is declared in the `for` initializer and is not accessible in the following block. Using `i` there causes a compilation error.
- Deallocation mismatch: Memory allocated with `new[]` must be released with `delete[]`. Using scalar `delete` with an array allocation causes undefined behavior.

**2. Corrected C++ code**

```
#include <iostream>

int main() {
    int size = 5;
    int *readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }

    std::cout << "Total: " << total << std::endl;

    delete[] readings;
    return 0;
}
```

**3. Dry run and output**

| i | Assigned reading | Running total |
|---|---|---|
| 0 | 10 | 10 |
| 1 | 20 | 30 |
| 2 | 30 | 60 |
| 3 | 40 | 100 |
| 4 | 50 | 150 |

```
Total: 150
```

# SECTION B: PSEUDOCODE WRITING QUESTIONS

Each pseudocode question is presented with its topic, scenario, student tasks, structured pseudocode answer, and sample-input trace table.

## Question 1: Procedural-to-OOP Paradigm Transformation

**Topic:** OOP paradigm, encapsulation, data hiding, abstraction.

Scenario: A municipal water authority wants a `WaterConsumer` class. Private attributes are consumerID (STRING), previousReading, currentReading, unitsConsumed and totalBill (REAL). Public methods are Initialize(id, prev, curr), CalculateConsumption(), ComputeBill(), and DisplayBill(). Readings must be non-negative and currentReading must be at least previousReading. Tariff: first 100 units at $1.50; next 200 units at $2.50; above 300 units at $4.00, plus a 10% environmental surcharge on the entire bill.

**Student Tasks:**

1. Write formal structured pseudocode defining WaterConsumer and an executable Main routine.
2. Trace previousReading = 1450.0 and currentReading = 1820.0, showing tier breakdown, surcharge and final total.

**ANSWER**

The following design gives the structured pseudocode and verifies it with the required sample-input trace.

**1. Design overview**

- Encapsulate consumer details and billing values as PRIVATE attributes.
- Expose PUBLIC methods to initialize and validate readings, calculate consumption, compute the tiered bill, and display the invoice.
- Consumption is current reading minus previous reading.
- Apply the surcharge only when consumption is above 300 units, as stated in the question.

**2. Structured pseudocode**

```
CLASS WaterConsumer
PRIVATE:
   consumerID : STRING
   previousReading : REAL
   currentReading : REAL
   unitsConsumed : REAL
   totalBill : REAL
   valid : BOOLEAN

PUBLIC:
   METHOD Initialize(id : STRING, prev : REAL, curr : REAL)
      IF prev < 0 OR curr < 0 OR curr < prev THEN
         valid <- FALSE
         DISPLAY "Invalid meter readings"
      ELSE
         consumerID <- id
         previousReading <- prev
         currentReading <- curr
         unitsConsumed <- 0
         totalBill <- 0
         valid <- TRUE
      ENDIF
   ENDMETHOD

   METHOD CalculateConsumption()
      IF valid = TRUE THEN
         unitsConsumed <- currentReading - previousReading
      ENDIF
   ENDMETHOD

   METHOD ComputeBill()
      DECLARE bill : REAL
      DECLARE surcharge : REAL
```

```
        IF valid = FALSE THEN
            RETURN
        ENDIF

        bill <- 0

        IF unitsConsumed <= 100 THEN
            bill <- unitsConsumed * 1.50
        ELSE
            bill <- 100 * 1.50
            IF unitsConsumed <= 300 THEN
                bill <- bill + (unitsConsumed - 100) * 2.50
            ELSE
                bill <- bill + 200 * 2.50
                bill <- bill + (unitsConsumed - 300) * 4.00
                bill <- bill + (bill * 0.10)
            ENDIF
        ENDIF

        totalBill <- bill
    ENDMETHOD

    METHOD DisplayBill()
        IF valid = TRUE THEN
            DISPLAY "Consumer ID: ", consumerID
            DISPLAY "Units Consumed: ", unitsConsumed
            DISPLAY "Total Bill: $", totalBill
        ENDIF
    ENDMETHOD
ENDCLASS

BEGIN
    DECLARE consumer : WaterConsumer
    consumer.Initialize("WC101", 1450.0, 1820.0)
    consumer.CalculateConsumption()
    consumer.ComputeBill()
    consumer.DisplayBill()
END
```

**3. Dry-run trace**

| Calculation | Units / formula | Amount |
|---|---|---|
| Consumption | 1820 - 1450 = 370 units | 370 |
| First tier | 100 × $1.50 | $150.00 |
| Second tier | 200 × $2.50 | $500.00 |
| Third tier | 70 × $4.00 | $280.00 |
| Subtotal | 150 + 500 + 280 | $930.00 |
| Surcharge | 10% × 930 | $93.00 |
| Final bill | 930 + 93 | $1,023.00 |

Expected final bill: $1,023.00.

## Question 2: Multi-Tier Tax & Precision Cast Engine

**Topic:** Control structures (selection), data types, type casting.

Scenario: Design an enterprise payroll tax algorithm. Inputs: grossAnnualSalary (REAL), deduction80C (REAL, cap $150,000), deductionHealthInsurance (REAL, cap $25,000). Taxable income is gross less eligible deductions, clamped to zero. Tax brackets: up to $250,000 at 0%; $250,001–$500,000 at 5% on excess over $250,000; $500,001–$1,000,000 at 20% on excess over $500,000; above $1,000,000 at 30% on excess over $1,000,000. Add 4% cess to raw tax. Convert final amount to INTEGER using nearest-rupee rounding equivalent to `static_cast<long>(tax + 0.5)`.

**Student Tasks:**

3. Write structured pseudocode for TaxCalculationEngine.
4. Trace grossAnnualSalary = $1,200,000, deduction80C = $180,000 and deductionHealthInsurance = $30,000.

**ANSWER**

The following design gives the structured pseudocode and verifies it with the required sample-input trace.

**1. Design overview**

- Cap each deduction at its permitted maximum.
- Subtract eligible deductions from gross salary and clamp taxable income to zero.
- Calculate tax progressively by applying each rate only to the income in that bracket.
- Add 4% cess to raw tax, then round to the nearest whole rupee.

**2. Structured pseudocode**

```
CLASS TaxCalculationEngine
PRIVATE:
   grossAnnualSalary : REAL
   deduction80C : REAL
   deductionHealthInsurance : REAL
   taxableIncome : REAL
   rawTax : REAL
   cess : REAL
   finalTax : REAL

PUBLIC:
   METHOD Calculate(gross : REAL, d80C : REAL, health : REAL)
      grossAnnualSalary <- gross
      deduction80C <- MIN(d80C, 150000)
      deductionHealthInsurance <- MIN(health, 25000)

      taxableIncome <- grossAnnualSalary
                     - deduction80C
                     - deductionHealthInsurance

      IF taxableIncome < 0 THEN
         taxableIncome <- 0
      ENDIF

      rawTax <- 0

      IF taxableIncome > 250000 THEN
         IF taxableIncome <= 500000 THEN
            rawTax <- (taxableIncome - 250000) * 0.05
         ELSE
            rawTax <- 250000 * 0.05
            IF taxableIncome <= 1000000 THEN
               rawTax <- rawTax
                       + (taxableIncome - 500000) * 0.20
            ELSE
               rawTax <- rawTax + 500000 * 0.20
               rawTax <- rawTax
                       + (taxableIncome - 1000000) * 0.30
            ENDIF
         ENDIF
      ENDIF

      cess <- rawTax * 0.04
      finalTax <- INTEGER(rawTax + cess + 0.5)
   ENDMETHOD

   METHOD DisplayResult()
      DISPLAY "Taxable Income: ", taxableIncome
      DISPLAY "Raw Tax: ", rawTax
```

```
        DISPLAY "Cess: ", cess
        DISPLAY "Final Tax: ", finalTax
    ENDMETHOD
ENDCLASS

BEGIN
    DECLARE engine : TaxCalculationEngine
    engine.Calculate(1200000, 180000, 30000)
    engine.DisplayResult()
END
```

**3. Dry-run trace**

| Step | Calculation | Value |
|---|---|---|
| 80C eligible deduction | MIN(180,000, 150,000) | $150,000 |
| Health deduction | MIN(30,000, 25,000) | $25,000 |
| Taxable income | 1,200,000 - 150,000 - 25,000 | $1,025,000 |
| 0% bracket | First $250,000 × 0% | $0 |
| 5% bracket | $250,000 × 5% | $12,500 |
| 20% bracket | $500,000 × 20% | $100,000 |
| 30% bracket | $25,000 × 30% | $7,500 |
| Raw tax | 12,500 + 100,000 + 7,500 | $120,000 |
| Cess | 120,000 × 4% | $4,800 |
| Final rounded tax | 120,000 + 4,800 | $124,800 |

Expected final tax: $124,800.

## Question 3: Bitwise Mask Validation & Sensor Health Telemetry

**Topic:** Operators in C++ (bitwise AND, OR, XOR, NOT), tokens, constants.

Scenario: An 8-bit unsigned UAV status register uses bits 0–7 for gyro locked (0x01), accelerometer calibrated (0x02), altimeter online (0x04), GPS fix (0x08), low battery (0x10), over-temperature (0x20), radio link (0x40), and fatal actuator failure (0x80). Mandatory bits 0–3 must be 1. Inhibitor bits 4, 5 and 7 must be 0. Radio bit 6 is non-blocking. Use bitwise masks without looping over bit positions. Report each inhibitor and use XOR to identify missing mandatory sensors.

**Student Tasks:**

5. Define the required hexadecimal bitmask constants.
6. Write structured pseudocode for CheckFlightReadiness(statusRegister).
7. Trace test bytes 0x4F and 0x3B.

**ANSWER**

The following design gives the structured pseudocode and verifies it with the required sample-input trace.

**1. Design overview**

- Use a mandatory mask 0x0F for bits 0–3 and inhibitor mask 0xB0 for bits 4, 5 and 7.
- Use bitwise AND to test required and inhibitor bits.
- Use XOR between the mandatory mask and the masked register bits to identify missing mandatory sensor bits.
- Do not treat radio bit 6 as a flight inhibitor.

**2. Structured pseudocode**

```
CONSTANT GYRO_MASK : INTEGER <- 0x01
CONSTANT ACCEL_MASK : INTEGER <- 0x02
CONSTANT ALTIMETER_MASK : INTEGER <- 0x04
CONSTANT GPS_MASK : INTEGER <- 0x08
CONSTANT LOW_BATTERY_MASK : INTEGER <- 0x10
CONSTANT OVER_TEMP_MASK : INTEGER <- 0x20
```

```
CONSTANT RADIO_MASK : INTEGER <- 0x40
CONSTANT FATAL_MASK : INTEGER <- 0x80

CONSTANT MANDATORY_MASK : INTEGER <- 0x0F
CONSTANT INHIBITOR_MASK : INTEGER <- 0xB0

METHOD CheckFlightReadiness(statusRegister : INTEGER)
   DECLARE activeMandatory : INTEGER
   DECLARE missingMask : INTEGER
   DECLARE activeInhibitors : INTEGER
   DECLARE ready : BOOLEAN

   activeMandatory <- statusRegister AND MANDATORY_MASK
   missingMask <- MANDATORY_MASK XOR activeMandatory
   activeInhibitors <- statusRegister AND INHIBITOR_MASK
   ready <- TRUE

   IF (activeInhibitors AND LOW_BATTERY_MASK) <> 0 THEN
      DISPLAY "ERROR: Low battery warning"
      ready <- FALSE
   ENDIF

   IF (activeInhibitors AND OVER_TEMP_MASK) <> 0 THEN
      DISPLAY "ERROR: Over-temperature warning"
      ready <- FALSE
   ENDIF

   IF (activeInhibitors AND FATAL_MASK) <> 0 THEN
      DISPLAY "ERROR: Fatal actuator failure"
      ready <- FALSE
   ENDIF

   IF missingMask <> 0 THEN
      DISPLAY "Missing mandatory sensor mask: ", HEX(missingMask)
      ready <- FALSE
   ENDIF

   IF ready = TRUE THEN
      DISPLAY "Flight Ready"
   ELSE
      DISPLAY "Flight NOT Ready"
   ENDIF
ENDMETHOD
```

**3. Trace table**

| Register | Mandatory bits present | Missing mask | Inhibitors | Decision |
|---|---|---|---|---|
| 0x4F | 0x0F (all four present) | 0x00 | 0x00 | Flight Ready; radio bit 6 is allowed. |
| 0x3B | 0x0B (bit 2 absent) | 0x04 | 0x30 (bits 4 and 5) | Not ready: low battery, over-temperature and altimeter missing. |

For 0x3B, the active inhibitor mask is 0x30, so both bit 4 (low battery) and bit 5 (over-temperature) must be reported. The missing mandatory sensor mask is 0x04 (barometric altimeter).

## Question 4: Optimized Prime Factorization & Divisor Engine

**Topic:** Control structures (for/while iteration; break, continue and goto jump statements).

Scenario: Accept a positive integer N, repeat input if N <= 0 using a jump/loop construct, handle N = 1 explicitly, generate proper divisors in O(sqrt(N)) by testing d from 2 to floor(sqrt(N)), use CONTINUE when d does not divide N,

and add paired divisors without double-counting a square root. Classify by sum of proper divisors as Perfect, Abundant or Deficient.

**Student Tasks:**

8. Write structured pseudocode for DivisorAndPrimeEngine.
9. Trace N = 28 and N = 12, showing d, N/d and sumOfDivisors at each step.

**ANSWER**

The following design gives the structured pseudocode and verifies it with the required sample-input trace.

**1. Design overview**

- Validate input until N is positive; handle N = 1 separately.
- Initialize the proper-divisor sum to 1 for N > 1.
- Check possible divisors only up to floor square root of N.
- If d divides N, add d and its paired divisor N/d; add the pair only once when both are equal.
- Compare the final sum with N to classify the number.

**2. Structured pseudocode**

```
BEGIN
   DECLARE N : INTEGER
   DECLARE d : INTEGER
   DECLARE pairedDivisor : INTEGER
   DECLARE sumOfDivisors : INTEGER

INPUT_AGAIN:
   DISPLAY "Enter a positive integer: "
   INPUT N

   IF N <= 0 THEN
      DISPLAY "Invalid input. Enter a positive integer."
      GOTO INPUT_AGAIN
   ENDIF

   IF N = 1 THEN
      DISPLAY "Deficient Number"
      DISPLAY "Sum of proper divisors: 0"
   ELSE
      sumOfDivisors <- 1
      d <- 2

      WHILE d * d <= N DO
         IF N MOD d <> 0 THEN
            d <- d + 1
            CONTINUE
         ENDIF

         pairedDivisor <- N / d
         sumOfDivisors <- sumOfDivisors + d

         IF pairedDivisor <> d THEN
            sumOfDivisors <- sumOfDivisors + pairedDivisor
         ENDIF

         d <- d + 1
      ENDWHILE

      IF sumOfDivisors = N THEN
         DISPLAY "Perfect Number"
      ELSE IF sumOfDivisors > N THEN
         DISPLAY "Abundant Number"
      ELSE
         DISPLAY "Deficient Number"
      ENDIF
```

```
    DISPLAY "Sum of proper divisors: ", sumOfDivisors
  ENDIF
END
```

**3. Dry-run trace: N = 28**

| d | 28 MOD d | Paired divisor (28/d) | Sum after step |
|---|---|---|---|
| Initial | — | — | 1 |
| 2 | 0 | 14 | 1 + 2 + 14 = 17 |
| 3 | Not 0 | — | 17 (unchanged) |
| 4 | 0 | 7 | 17 + 4 + 7 = 28 |
| 5 | Not 0 | — | 28 (unchanged) |

Loop stops after d exceeds floor(sqrt(28)) = 5. Sum = 28, so 28 is a Perfect Number.

**4. Dry-run trace: N = 12**

| d | 12 MOD d | Paired divisor (12/d) | Sum after step |
|---|---|---|---|
| Initial | — | — | 1 |
| 2 | 0 | 6 | 1 + 2 + 6 = 9 |
| 3 | 0 | 4 | 9 + 3 + 4 = 16 |

Loop stops after d exceeds floor(sqrt(12)) = 3. Sum = 16, so 12 is an Abundant Number.

## Question 5: State-Machine ATM Controller with Sentinel Loops

**Topic:** Control structures (`switch-case`, `while`, nested `if-else`), data types.

Scenario: Design an ATM transaction manager. Preset PIN is 4821; allow up to three attempts and lock the card after three incorrect attempts. After authentication, repeat a menu: 1 balance inquiry, 2 deposit, 3 withdrawal, 4 exit. Deposits must be positive. Withdrawals must be positive, no greater than balance, and exact multiples of 100. Invalid options must be handled.

**Student Tasks:**

10. Write formal structured pseudocode for ATM_SessionController.
11. Trace one wrong PIN followed by the correct PIN, a declined $250 withdrawal, and a successful $300 withdrawal.

**ANSWER**

The following design gives the structured pseudocode and verifies it with the required sample-input trace.

**1. Design overview**

- Authenticate with a maximum of three PIN attempts; stop the session immediately if all attempts fail.
- After successful authentication, use a sentinel-controlled menu loop that ends when the user selects Exit.
- Use SWITCH to process menu options and nested IF statements to validate deposits and withdrawals.
- Since the question does not specify an opening balance, the dry run below assumes an initial balance of $1,000.

**2. Structured pseudocode**

```
METHOD ATM_SessionController()
  DECLARE correctPIN : INTEGER <- 4821
  DECLARE enteredPIN : INTEGER
  DECLARE attempts : INTEGER <- 0
  DECLARE authenticated : BOOLEAN <- FALSE
  DECLARE balance : REAL <- 1000.00
  DECLARE choice : INTEGER
  DECLARE amount : REAL

  WHILE attempts < 3 AND authenticated = FALSE DO
    DISPLAY "Enter PIN: "
```

```
      INPUT enteredPIN
      attempts <- attempts + 1

      IF enteredPIN = correctPIN THEN
         authenticated <- TRUE
         DISPLAY "Authentication successful"
      ELSE
         DISPLAY "Incorrect PIN"
      ENDIF
   ENDWHILE

   IF authenticated = FALSE THEN
      DISPLAY "Card locked. Too many incorrect attempts."
      RETURN
   ENDIF

   choice <- 0
   WHILE choice <> 4 DO
      DISPLAY "1. Balance Inquiry"
      DISPLAY "2. Cash Deposit"
      DISPLAY "3. Cash Withdrawal"
      DISPLAY "4. Exit"
      INPUT choice

      SWITCH choice
         CASE 1:
            DISPLAY "Available balance: $", balance
            BREAK

         CASE 2:
            DISPLAY "Enter deposit amount: "
            INPUT amount
            IF amount > 0 THEN
               balance <- balance + amount
               DISPLAY "Deposit successful"
            ELSE
               DISPLAY "Invalid deposit amount"
            ENDIF
            BREAK

         CASE 3:
            DISPLAY "Enter withdrawal amount: "
            INPUT amount
            IF amount <= 0 THEN
               DISPLAY "Invalid withdrawal amount"
            ELSE IF amount > balance THEN
               DISPLAY "Insufficient balance"
            ELSE IF amount MOD 100 <> 0 THEN
               DISPLAY "Declined: Amount must be a multiple of 100"
            ELSE
               balance <- balance - amount
               DISPLAY "Withdrawal successful"
               DISPLAY "Remaining balance: $", balance
            ENDIF
            BREAK

         CASE 4:
            DISPLAY "Thank you. Session ended."
            BREAK

         DEFAULT:
            DISPLAY "Invalid menu option"
      ENDSWITCH
   ENDWHILE
ENDMETHOD
```

**3. Dry-run trace**

| Step | Input / action | System response | Balance |
|---|---|---|---|
| 1 | PIN = 1111 | Incorrect PIN; attempt 1 of 3 | $1,000 |
| 2 | PIN = 4821 | Authentication successful; menu shown | $1,000 |
| 3 | Select 3; amount = $250 | Declined: not a multiple of 100 | $1,000 |
| 4 | Select 3; amount = $300 | Withdrawal successful | $700 |
| 5 | Select 4 | Thank you. Session ended. | $700 |

The user is authenticated on the second attempt. The $250 withdrawal is rejected because 250 MOD 100 is not zero. The $300 withdrawal is accepted, leaving $700.

# Important Notes

- All debugging programs are complete corrected examples, not isolated replacement lines.
- All pseudocode answers use the question's stated requirements and sample inputs.
- Question B5 uses an assumed opening balance of $1,000 only because the source question does not specify one.
- Question B3: test byte 0x3B activates both low-battery (bit 4) and over-temperature (bit 5) inhibitors, and it is missing mandatory bit 2.